

多目标基线

我们目前在做这方面的工作（见pdf），审稿人给我们的意见是：对比的算法比较简单，不足以体现方法先进性，To demonstrate true effectiveness, the proposed algorithm must be evaluated against **recent state-of-the-art multi-objective optimization algorithms or modern learning-based deployment strategies**.

你能不能帮我调研一下相关论文，帮我找一找对比算法，作为我们实验部分的对比基线？最好还有源代码仓库这样我们方便复现。

请你在回复中，回复包含你阅读的论文文献（论文标题）以及他的算法简介，如果有仓库链接请你附上。

1 MTDMS-CMOEA

Tuo S, Li C, Huyan Y, et al. MTDMS-CMOEA: A Multi-Task dynamic membrane computing framework for Tri-Objective optimization of Latency, Energy, and security in IoT edge offloading[J]. Expert Systems with Applications, 2026, 310: 131237.

2026年国际顶级期刊《Expert Systems With Applications》

源代码: [shouhengtuo/MTDMS-CMOEA](#)

1.1 问题

随着物联网设备的普及，很多计算密集型的任务受限于设备自身的算力和电池，必须“卸载（Offload）”到边缘服务器上处理。然而，在实际应用中，规划这个卸载过程面临着一个巨大的挑战：

- 三个相互冲突的目标：系统需要同时最小化延迟（**Latency**）、能耗（**Energy**）和安全风险（**Security**）。比如，为了省电把任务传到边缘服务器会增加传输延迟，而使用高级别加密来保证安全又会额外增加计算时间和能耗。
- 高维且强约束：系统不仅变量极其庞大（涉及决定去哪卸载、分配多少资源、是否缓存、用什么加密级别），还要满足非常死板的“硬约束”（如任务死线、服务器容量、缓存上限等）。传统的优化算法和深度强化学习（DRL）在这个问题上面临维度灾难，极难找到甚至一个可行解。

1.2 核心方法：MTDMS-CMOEA

为了解决上述痛点，作者提出了一种名为 **MTDMS-CMOEA** 的新算法。这是一个基于“膜计算（**Membrane Computing**）”结构的多任务动态约束多目标进化算法。其最大创新在于三阶段解耦优化机制：

论文作者发现，安全策略的分配是可以独立出来的，它不会改变基础的任务卸载去向和资源分配。因此，他们没有像传统方法那样把三个目标混在一起优化，而是分成了三步走：

- 第一阶段：多任务并行探索（**Multi-Task Exploration**）。利用生物细胞“膜”的概念，将种群划分到4个不同的“膜”中并行进化，分别去探索约束双目标、无约束双目标、单纯追求极低延迟和单纯追求极低能耗的方向。这样保证了搜索的多样性。
- 第二阶段：加速收敛（**Accelerated Convergence**）。利用动态的约束处理机制，迫使算法快速从庞大的搜索空间中找到那些满足所有严苛条件的“可行解”（即既不超时也不超载的方案）。这两个阶段锁定了高质量的“延迟-能耗”平衡点。
- 第三阶段：安全感知微调（**Security-Aware Refinement**）。在不改变前两阶段核心卸载决定的前提下，单独作为一个轻量级的后处理步骤来调整安全级别（如选择AES-128还是AES-256）。这样既最大化了安全性，又避免了高维多目标优化带来的灾难。

1.3 其他创新点

- 将服务器缓存（**Caching**）作为核心变量纳入模型：通过智能判断哪些任务的数据可以提前缓存在边缘服务器，显著减少了因为加密传输带来的通信负担。

1.4 实验结果与贡献

作者在标准测试集（DOC）和大规模真实物联网场景（每个节点多达50个设备，约420个决策变量）中进行了广泛测试：

- 性能碾压基准算法：在收敛性、多样性和保证100%找到可行解方面，持续优于现有的前沿进化算法（ICMA, IMTCMO）以及基于DRL（强化学习）的方法。
- 极低的安全开销：由于采用了巧妙的第三阶段微调，算法使得安全风险大幅降低了**37.8%–57.9%**，但付出的代价极小（仅增加了约14.2%的延迟，能耗甚至有-0.1%的微小优化）。而引入缓存机制让与安全相关的传输开销降低了18.7%。
- 高并行性与实时性：由于“膜计算”天然的并行优势，算法在普通的6核CPU上实现了近**3倍**的并行加速，且面对动态的安全威胁，系统能在**10秒**内完成策略的重新计算与更新。

1.5 MTDMS-CMOEA Algorithm（基于多任务动态膜结构的约束多目标进化算法）

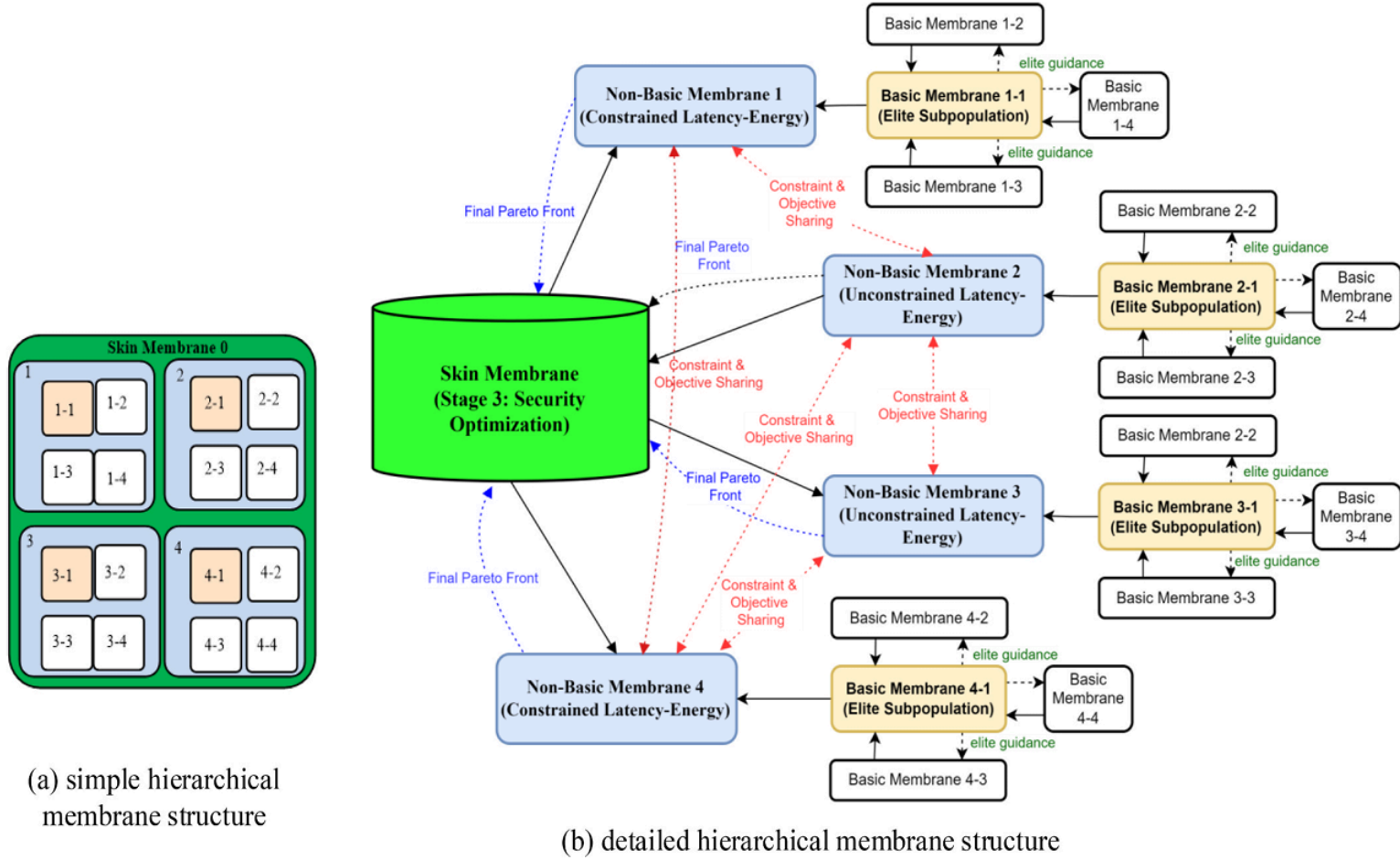


Fig. 2. Hierarchical Membrane Structure of MTDMS-CMOEA. (a) Simple view showing the three-layer hierarchy. (b) Detailed view of the four specialized non-basic membranes.

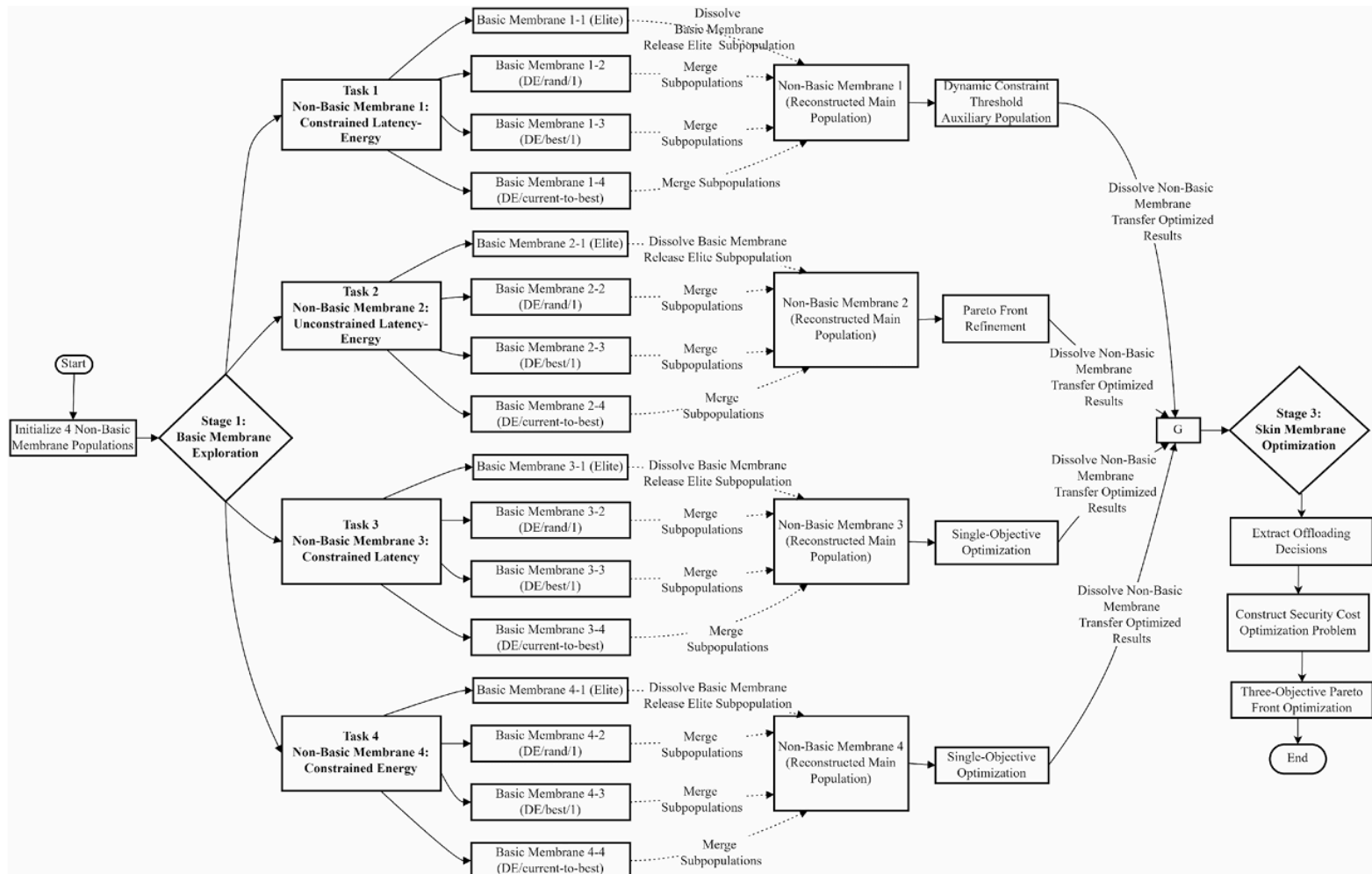


Fig. 3. The flowchart of MTDMS-CMOEA Algorithm.

1.6 MTDMS-CMOEA 算法主框架 (Algorithm 1: Main Framework)

首先生成四个子种群并将其分配至层级化的膜结构中，随后程序进入第一阶段的多任务探索，利用 Algorithm 2 在四个膜内并行演化。

第二阶段，针对初步筛选出的个体调用 Algorithm 3，利用动态约束处理机制引导种群快速向可行域收敛，并在演化结束后溶解所有非基础膜实现种群大合并。

最后在第三阶段，算法通过非支配排序提取出精英可行解。

Input : $N, D, \max FEs$

输入：种群大小 N ，问题维度 D ，最大函数评估次数 $\max FEs$ 。这是算法运行的基础参数， N 决定了搜索方案的数量， D 代表决策变量的总数（如卸载决策和缓存决策的组合）， $\max FEs$ 则限制了总计算成本。

*Output : Pop^**

输出：帕累托最优解集 Pop^* 。算法最终返回一组在延迟、能耗和安全风险之间达到平衡的最优方案集合，用户可以根据实际偏好从中选择最合适的一个。

1. $Pop_1, Pop_2, Pop_3, Pop_4 \leftarrow \text{init_pop}(N, D)$

1. 初始化四个子种群 $Pop_1, Pop_2, Pop_3, Pop_4$ 。基于膜计算的思想，算法刚开始就将大种群平均分成四个独立的小组，每一组都随机生成一套初始方案，为后续在四个不同的“膜区域”进行并行进化做准备。

2. *// STAGE1 : Multi - Task Exploration*

2. // 第一阶段：多任务探索（在 4 个专业膜中并行进化）。这一阶段的目标是“广撒网”，利用膜计算的并行性，让不同的子种群去探索搜索空间的不同特征区域，从而保证解的多样性。

3. *while $FEs < \max FEs_{stage1}$ do*

3. 当当前评估次数小于第一阶段设定的最大次数时，执行循环。这是一个计算预算控制机制，确保算法在第一个阶段有足够的迭代次数来摸清问题的底细，直到达到预设的阶段评估上限。

4. *for each $i \in \{1, 2, 3, 4\}$ do*

4. 遍历每一个子种群编号 i （对应不同的优化任务）。算法对四个膜进行分工：膜 1 处理带约束的双目标优化，膜 2 负责无约束的全局探测，膜 3 和膜 4 分别专注于极速延迟和极低能耗的单目标搜索。

5. $Pop_i \leftarrow \text{Algorithm 2}(Pop_i, i)$

5. 调用算法 2 对第 i 个子种群进行并行进化演化。这是实际的变异、交叉和选择过程，利用差分进化（DE）算子让每个子种群在自己的“专业领域”内进化，产生更符合该膜目标的优秀个体。

6. *end for*

6. 结束当前的子种群遍历循环。这代表四个专业化的小组已经完成了一轮各自的进化任务。

7. *Dissolve basic membranes : Merge subpopulations*

7. 溶解基础膜：合并每个非基础膜内的子种群。这是一个受生物启发的操作，通过“溶解”膜让原本隔离的子种群进行信息交换，把各组找到的“好基因”汇总，提升整体的进化效率。

8. *end while*

8. 结束第一阶段的迭代循环。此时算法已经完成了初步的全局探索，手中掌握了一批在不同维度表现优异的方案雏形。

9. *// STAGE2 : Accelerated Convergence*

9. // 第二阶段：加速收敛（针对可行性的动态约束处理）。在有了第一阶段的探索基础后，这一阶段的目标是“收网”，重点解决那些违反了硬性约束（如超时、内存溢出）的方案，强迫它们回归到可行区域。

10. *for each $i \in \{1, 2, 3, 4\}$ do*

10. 遍历每一个子种群编号 i 。再次对合并后的群体进行分组处理，以便利用不同的约束处理策略进行精细化调整。

11. $Pop_i \leftarrow \text{Algorithm 3}(Pop_i)$

11. 调用算法 3 对子种群应用动态约束处理机制。该机制使用一种“由宽到严”的动态阈值方法，引导那些不达标的方案逐步演化成满足所有延迟和能耗限制的合法方案，显著提高寻找可行解的速度。

12.*end for*

12. 结束第二阶段的子种群处理循环。到这里为止，种群中的大部分个体应该已经成为了既高性能又符合现实约束的可行方案。

13. $Pop \leftarrow Pop_1 \cup Pop_2 \cup Pop_3 \cup Pop_4$

13. 溶解非基础膜：合并所有子种群得到总种群 **Pop**。将第二阶段所有优化后的子种群再次汇聚成一个大池子，准备进行最后的筛选和精雕细琢。

14.*//STAGE3 : Security – AwareRefinement*

14. // 第三阶段：安全感知精炼（仅对安全等级进行轻量化优化）。这是该算法的独创设计，将安全性从复杂的调度问题中剥离出来，在不改变任务卸载位置和资源分配的前提下，只优化加密协议的选择。

15. $Pop \leftarrow nonDominatedSorting(Pop)$

15. 对总种群进行非支配排序，获取可行的精英解。通过数学筛选，剔除掉那些在延迟和能耗上都比别人差的方案，只保留那些表现出色、无法被轻易替代的精英个体作为待精炼对象。

16. $Pop \leftarrow Algorithm\ 4(Pop)$

16. 调用算法 4 进一步优化精英解的安全等级分配。在这些精英方案的基础上，针对每个任务快速匹配最优的安全策略，从而在不增加延迟和能耗负担的情况下，大幅降低整个系统的安全风险值。

17.*return Pop*

17. 返回最终的帕累托最优解集。算法运行结束，输出一组经过三阶段打磨的、在延迟、能耗和安全这三个维度都表现优异的权衡方案供决策使用。

1.7 第一阶段：多任务并行探索 (Algorithm 2: Multi-Task Exploration)

我们首先会根据适应度表现将子膜种群精细化地拆解为精英组、随机组、最佳组和混合组这四个各占 25% 的功能子模块。

在接下来的迭代循环中，每个小组会根据自身的定位采用不同的差分进化算子，比如精英组利用 DE/best/1 策略深度挖掘优质区域，而随机组和最佳组则通过 DE/rand/1 等方式维持探索强度。

随后算法会根据当前子膜承载的具体任务来分别进行适应度评估，在完成每轮进化后，我们通过精英保留机制确保种群质量稳步提升，并特别引入了“精英引导”机制。

最后在消耗完预设的计算预算后，将这四个各有所长的子组合并输出，为后续阶段提供一个具备强大进化潜力的候选解集。

1.*Partition $\mathcal{P}^{(m)}$ into four subgroups : (1) Elite; (2) RandGroup; (3) BestGroup; (4) HybridGroup*

1. 将当前子膜的种群划分为四个子组：精英组、随机组、最佳组和混合组。为了在搜索过程中兼顾局部深度挖掘和全局广度探索，算法将个体按 25% 的比例拆成四个功能小组，精英组保留最强方案，其他小组则从不同维度寻找新可能。

2. $FE \leftarrow 0$

2. 初始化函数评估次数计数器为 0。该变量用于追踪算法在当前阶段已经消耗了多少计算资源，以便在达到预算上限时及时停止迭代。

3.*while $FE < FE_{max}^{Stage1}$ do*

3. 当函数评估次数未达到第一阶段的预算上限时持续循环。这是迭代控制的核心逻辑，确保子膜在限定的计算资源内尽可能多地进行进化搜索。

6.*Generate mutant via DE/best/1 : $\mathbf{v} = \mathbf{x}_{best} + F \cdot (\mathbf{x}_{r1} - \mathbf{x}_{r2})$*

4. 通过 DE/best/1 策略为精英组生成变异向量。精英组采用最激进的“贪婪”策略，它始终以当前子膜中最优秀的个体 ($\mathbf{x}_{best}$) 为基准，通过叠加其他方案的差异来尝试微调，旨在快速榨取当前最优区域的价值。

7. *Apply crossover to produce trial vector $\mathbf{u}$*

5. 执行交叉操作以产生实验向量 $\mathbf{u}$ 。通过将变异后的方案与原始方案按一定概率交换基因，增加了解的随机性，防止搜索过程陷入僵化的单一方向。

11. *Generate mutant via DE/rand/1 : $\mathbf{v} = \mathbf{x}_{r1} + F \cdot (\mathbf{x}_{r2} - \mathbf{x}_{r3})$*

6. 为随机组通过 DE/rand/1 策略生成变异向量。随机组不迷信当前的最强者，而是完全从三个随机个体中提取差异信息，这种“盲目”的探索方式能有效维持种群多样性，防止算法过早被困在局部陷阱中。

14. *Generate mutant via DE/current - to - best/1 : $\mathbf{v} = \mathbf{x} + F \cdot (\mathbf{x}_{best} - \mathbf{x}) + F \cdot (\mathbf{x}_{r1} - \mathbf{x}_{r2})$*

7. 为最佳组通过当前向最佳靠拢策略生成变异向量。这是一种平衡型策略，它在保持个体自身特色的基础上，试图向最强者靠拢，非常适合在探索未知区域和利用已知信息之间寻找中间点。

18. *Use a hybrid DE strategy (e.g., ensemble of above)*

8. 为混合组采用混合差分进化策略。混合组通过集成多种变异模式，根据搜索状态灵活调整，从而应对复杂的决策空间。

21. *for each trial vector $\mathbf{u}$ from all groups do*

9. 遍历所有小组生成的实验向量。算法准备对这一轮进化出的所有新方案进行质量评估。

22 – 23. *if $m = 1$ then Evaluate with constrained bi - objective (T_{total}, E_{total}) + constraints*

10. 若当前为膜 1，则评估带约束的延迟与能耗双目标。这是最复杂的任务，需要同时计算系统总延迟和总能效，并严格检查是否满足截止日期和服务器容量等硬性约束。

24 – 25. *else if $m = 2$ then Evaluate with unconstrained bi - objective (T_{total}, E_{total})*

11. 若为膜 2，则仅评估延迟与能耗双目标，不考虑约束。该任务作为“探路者”，旨在摸清不带限制时的理论最优边界，为其他膜提供进化方向的参考。

26 – 27. *else if $m = 3$ then Evaluate with single - objective latency T_{total} + constraints*

12. 若为膜 3，则评估带约束的单目标延迟。该膜作为“测速专家”，专攻如何将系统响应速度提到极限。

28 – 29. *else if $m = 4$ then Evaluate with single - objective energy E_{total} + constraints*

13. 若为膜 4，则评估带约束的单目标能耗。该膜作为“节油专家”，专注于寻找最省电的卸载方案。

33 – 36. *Subgroup $\leftarrow$ Select top 25% from $\{Subgroup \cup offspring\}$*

14. 通过精英保留机制更新各个子组。将旧的个体与新生成的后代放在一起进行“海选”，只保留表现最优秀的前 25%，确保种群在每一代都在不断进化。

37 – 40. *Elite Guidance : With probability p_{elite} , use $\mathbf{x}_{best} \in Elite$ as one parent in mutation*

15. 精英引导：以一定概率让探索组在变异时参考精英组的优秀方案。这是膜内部的高效协同机制，让那些在外面“瞎逛”的小组偶尔参考一下精英的经验，从而在大雾中找到大致的优化方向，加速整体收敛。

43. *$FE \leftarrow FE + number\ of\ new\ evaluations$*

16. 更新已消耗的函数评估次数。将本轮新增的计算成本累加到计数器中。

45. *Merge Elite, RandGroup, BestGroup, HybridGroup into $\mathcal{P}_{out}^{(m)}$*

17. 将四个子组合并为最终的输出种群。完成迭代后，四个小组带着各自领域学到的经验再次合体，形成一个更强大的整体，为后续阶段提供高质量的基础。

46. *return $\mathcal{P}_{out}^{(m)}$*

18. 返回优化后的子种群。算法 2 任务完成，将成果提交给主框架。

1.8 第二阶段：约束动态加速收敛 (Algorithm 3: Accelerated Convergence)

算法三核心任务：实现从广域探索到严格可行域的快速收敛。

采用了主、辅双种群协同进化的策略，通过一个随时间递减的动态阈值 τ 来管理对约束违规的容忍度。

流程上首先继承第一阶段的种群并将其克隆为主攻性能的主种群和保留潜力的辅助种群，在每一轮迭代中，对主、辅种群分别应用差分进化算子生成后代。两个种群在各自的范围内演变，并不断产生新的候选方案，其中主种群注重性能提升，辅助种群注重在约束边界附近徘徊寻找机会。

紧接着根据当前时刻的 τ 值来执行严苛的生存筛选，筛选规则遵循可行解绝对支配违规解的原则，随着迭代进行，这个容忍阈值最终会归零，从而像漏斗一样强制把所有个体挤进满足截止日期、容量等所有硬性指标的可行空间。

Input : $\mathcal{P}_{in}, FE_{max}^{Stage2}, \tau_0$

输入：阶段 1 继承的种群、第二阶段评估预算、初始约束违反阈值。这一阶段需要承接前期的探索成果，并引入一个初始阈值来管理算法对方案违规程度的容忍度。

1 – 2. $\mathcal{P}_{main} \leftarrow \mathcal{P}_{in}, \mathcal{P}_{aux} \leftarrow \mathcal{P}_{in}$

1-2. 初始化主种群和辅助种群。算法采用双种群并行机制，主种群负责向最优性能冲刺，辅助种群则负责在不可行区域保留多样性，避免算法过早陷入狭小的搜索盲区。

7. $\tau(t) = \tau_0 \cdot (1 - t/t_{max})$

7. 更新动态约束违反阈值。这是一个随时间线性衰减的控制函数，初期阈值较大，允许方案适度违反约束（如稍微超时）以寻找潜在捷径，后期阈值趋于零，严格强迫所有方案回归合法。

9 – 10. $\mathcal{Q}_{main} \leftarrow DE(\mathcal{P}_{main}), \mathcal{Q}_{aux} \leftarrow DE(\mathcal{P}_{aux})$

9-10. 对主、辅种群分别应用差分进化算子生成后代。两个种群在各自的范围内演变，并不断产生新的候选方案，其中主种群注重性能提升，辅助种群注重在约束边界附近徘徊寻找机会。

13. $CV(x) = \sum_{j=7}^{10} \max(0, LHS_j - RHS_j)$

13. 计算每个个体的总约束违反量。该步骤将公式 7 到 10 定义的所有硬性规则（如功耗、存储、带宽）的违规值求和，数值越小代表方案越接近现实可行，0 则代表完美合规。

16. $\mathcal{P}_{main} \leftarrow NonDominatedSelection(\mathcal{P}_{main} \cup \mathcal{Q}_{main} \cup \mathcal{Q}_{aux})$

16. 使用受约束支配原则更新主种群。在筛选下一代时，遵循可行方案绝对优于不可行方案的原则，若都可行则按延迟和能耗的平衡度排序，确保种群稳步向最优可行区域靠拢。

19. $\mathcal{P}_{aux} \leftarrow Select\ individuals\ with\ CV(x) \leq \tau(t)$

19. 筛选违规程度在容忍范围内的个体进入辅助种群。只要个体的违规量小于当前的动态阈值，就能存活下来，这些解往往携带了“高性能”的基因，是引导主种群跨越搜索障碍的关键桥梁。

23. *return* $\mathcal{P}_{main}$

23. 返回进化完成且富含高质量可行解的主种群。经过这一阶段的强力收敛，算法已经筛选出一批既能快速执行、又极度省电且绝对符合系统硬性规定的优质方案。

1.9 第三阶段：安全感知轻量级微调 (Algorithm 4: Security-Aware Refinement)

在论文中，解耦（Decoupling）指的是一种“决策层面的先后顺序”和“物理影响的单向性”。

你可以把任务卸载方案想象成“盖房子”：

- **Stage 1 & 2（核心决策）**：决定房子盖在哪里（卸载到哪个服务器）、盖多大空间（分配多少CPU）、要不要放个储藏室（缓存）。这些是“硬结构”，一旦动了，房子的造价（能耗）和入住时间（延迟）会发生剧变，甚至因为地基不稳（违反约束）导致房子塌了。
- **Stage 3（安全精炼）**：决定房子装什么防盗门、配什么指纹锁（安全等级）。

- 解耦的逻辑在于：作者发现，你把防盗门从A等级换成B等级，虽然会稍微增加一点安装时间（延迟）和买门的钱（能耗），但它绝对不会导致你房子的地基裂开（违反核心约束）。因为安全等级带来的开销是“模块化”且“叠加”的，它不会改变任务“该去哪”的最优逻辑。

所以，解耦意味着：我先定下房子的主体结构（最优卸载方案），然后再在这个基础上寻找最划算的安全配置。

如果你的目标是“紧耦合”的，那么不需要算法四，或者说算法四无法独立存在。

- 在这种情况下怎么办？
 1. 合并阶段：你不需要单独的 Stage 3。你需要在 Stage 1 和 Stage 2 中直接把所有目标（包括原本在 Stage 3 的目标）放在一起，作为一个完整的多目标问题来跑。
 2. 算法退化：此时 MTDMS-CMOEA 就退化成了一个带有并行膜结构的常规约束多目标算法。
- 结论：算法四是作者针对“安全防护”这个特定属性开发的“提速插件”。如果你的问题里没这种“可以最后再微调”的目标，那就直接在第一、二阶段把所有目标都解决了。

Input : $\mathcal{P}_{feas}, FE_{max}^{Stage3}$

输入：第二阶段获得的可行方案集、第三阶段的计算预算。算法利用前两个阶段已经确定的高性能卸载方案作为“蓝本”，在有限的预算内专门针对安全这一维度进行最后的“点睛之笔”。

1. *for each solution* $s = (\mathbf{x}, \mathbf{y}, \mathbf{f}) \in \mathcal{P}_{feas}$ *do*

1. 遍历每一个在第二阶段筛选出的精英方案。算法采取“各个击破”的策略，对每一个优质的卸载布局进行单独的安全增强，确保不会因为调整安全策略而破坏原有的延迟和能耗优势。

2. $\mathcal{O} \leftarrow \{(i, n) | x_{i,n} = 1\}$

2. 识别当前方案中所有已卸载的任务集合。由于只有卸载到边缘服务器的任务才涉及复杂的网络安全风险，算法通过这一步精确锁定需要进行安全精炼的对象，避免对本地执行的任务做无用功。

3. $D_z \leftarrow |\mathcal{O}|$

3. 将问题维度缩减为已卸载任务的数量。这是算法“轻量化”的关键，它不再去处理成千上万个原始变量，而是只关注这几十个安全等级开关，极大地降低了计算复杂度。

4. *Initialize population* $\mathcal{P}_z$ *with individuals encoding security levels* $z_{i,n}$

4. 初始化一个专门针对安全等级编码的小种群。为当前的卸载方案随机生成一组安全协议组合（如 AES-128、RSA 等），作为后续进化优化的起点。

7. *Generate offspring population* $\mathcal{Q}_z$ *from* $\mathcal{P}_z$ *using* $DE/rand/1$

5. 使用 DE/rand/1 策略生成新的安全配置后代。利用差分进化算法在可能的安全协议组合中进行搜索，尝试寻找在不增加太多开销的情况下风险更低的加密组合。

9. *Compute full objectives* : $T_{total}(\mathbf{x}, \mathbf{y}, \mathbf{f}, \mathbf{z}), E_{total}(\mathbf{x}, \mathbf{y}, \mathbf{f}, \mathbf{z}), \Psi(\mathbf{z})$

6. 计算包含三目标在内的完整评估值。因为改变安全等级会增加额外的计算延迟和能耗，算法在此步将这三者合并计算，正式将“安全风险”作为第三个优化目标引入评价体系。

11. $CV_{i,n} = \max(0, \phi_i^{min} - \phi(z_{i,n}))$

7. 计算安全合规性的约束违反量。检查选定的安全协议是否达到了任务要求的最低安全门槛，如果协议强度不够，该方案将被记录为违规。

14. $\mathcal{P}_z \leftarrow NonDominatedSelection(\mathcal{P}_z \cup \mathcal{Q}_z)$

8. 对安全方案进行非支配排序和拥挤度筛选。在延迟、能耗、安全风险这三个维度上挑选表现最均衡的配置，确保留下的都是在牺牲太多速度的前提下最安全的组合。

17. *Append the best individual from* $\mathcal{P}_z$ *to* $\mathcal{P}_{refined}$

9. 将每个蓝本方案对应的最佳安全配置添加到最终结果集中。经过这一步，原本只有两个目标的精英方案被赋予了最优的安全属性，升级为完整的三目标最优解。

19. *return* $\mathcal{P}_{refined}$

10. 返回精炼后的最终帕累托解集。至此，算法完成了从“粗加工”到“精加工”的全过程，输出了一组在延迟、能耗、安全三者之间达到艺术级平衡的决策方案。

2 METOA

Yang C, Chen Q, Zhu Z, et al. Evolutionary multitasking for costly task offloading in mobile-edge computing networks[J]. IEEE Transactions on Evolutionary Computation, 2023, 28(2): 338-352.

无开源代码。

2.1 问题定义

在移动边缘计算（MEC）网络中，资源受限的移动设备（如手机）可以将复杂的计算任务卸载（Offloading）到边缘服务器上处理，从而降低设备的能耗并缩短处理延迟。

但是，随着接入设备和计算任务呈指数级增长，如何决定哪些任务该卸载、分配多少计算资源、以及分配多少传输带宽/功率，变成了一个极度复杂的组合优化问题。作者将这一问题称为 **CTOP（Costly Task Offloading Problem，高代价任务卸载问题）**。直接求解这个庞大的问题（既要优化设备能耗，又要优化处理延迟）计算成本极高，传统算法很难在合理时间内找到最优解。

2.2 核心思想与创新点

为了高效解决CTOP，作者引入了演化多任务（**Evolutionary Multitasking, EMT**）的思想。EMT的核心理念是：在解决一个极其复杂的“目标任务”时，可以构造几个相关的、简单的“辅助任务”同步求解，并让它们互相“交流经验（知识转移）”，从而加速复杂任务的求解。

在此基础上，论文提出了一个全新的多任务演化任务卸载算法（**METOA**），主要创新点包括：

- **巧妙设计辅助任务（Auxiliary Tasks）**：作者将复杂的CTOP拆解并构造了两种简单廉价的辅助任务，这两种辅助任务计算量小且容易求解，其优化经验对决定最终的“卸载决策”极具指导意义。
- **分段知识转移策略（Segmented Knowledge Transfer）**：算法将决策变量划分为“源决策变量”和“扩展决策变量”。通过将辅助任务中得到的好基因（高质量解）分段转移给目标主任务。其中源变量用于加强局部搜索（加速收敛），扩展变量用于保持种群多样性（防止陷入局部最优）。
- **辅助任务动态更新机制（Auxiliary Task Update）**：因为辅助任务比较简单，容易早熟收敛。一旦辅助任务停滞，无法再为目标任务提供有用的“知识”时，算法会动态生成新的辅助任务。这不仅节约了计算资源，还能在进化的不同阶段持续为目标任务提供新的启发。

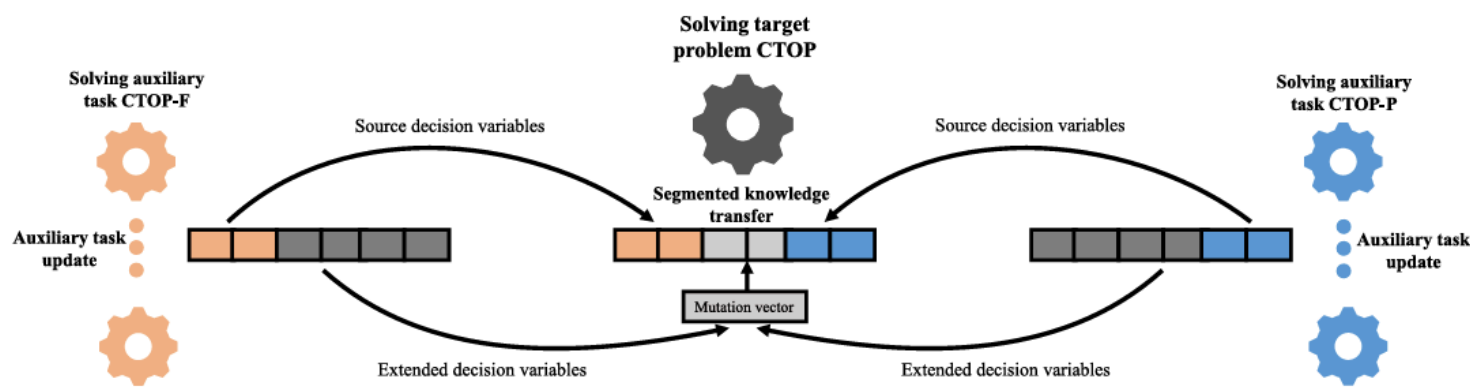


Fig. 3. Example of proposed METOA with segmented knowledge transfer and auxiliary tasks update.

2.3 算法 1 伪代码

输入 (Input):

Total population size - NP 总种群规模 NP

Target problem CTOP - TC 目标问题 TC

输出 (Output):

The optimized solution of CTOP 优化后的任务卸载方案

算法步骤:

1. Set $t = 1$ 设置初始代数为 1。
2. Initialize target population C^t for TC of size $NP/2$ 为目标问题初始化一个规模为 $NP/2$ 的种群 C^t 。
3. Evaluate candidate individuals in C^t 评估目标种群中所有候选个体的适应度。
4. Generate auxiliary tasks $TA = \{TA_j\}_{j=1}^2$ based on CTOP-F and CTOP-P 根据公式 10 和 11，分别生成两个辅助任务。
5. Initialize auxiliary populations $P^t = \{P_j^t\}_{j=1}^2$ for TA of size $NP/4$, respectively 为每个辅助任务初始化种群，每个辅助种群规模为 $NP/4$ 。
6. Evaluate candidate individuals in P^t 评估辅助种群中所有候选个体的适应度。
7. While stopping criteria are not satisfied do 当不满足停止条件（如达到最大评价次数）时，执行循环。
8. Segmented knowledge transfer via Algorithm 2 通过算法 2 执行分段知识迁移操作。
9. Uniformly divide the C^t into $\{C_j^t\}_{j=1}^2$ 将当前的目标种群均匀划分为两个子种群。
10. For $j = 1$ to 2 do 对每一个辅助任务及其对应的目标子种群开始内部循环。
11. Perform assortative mating on $C_j^t \cup P_j^t$ to generate offspring CP_j^t 在目标子种群与辅助种群的并集上执行同类交配，生成后代 CP_j^t 。
12. Determine the imitation task of offspring in CP_j^t based on vertical cultural transmission 基于垂直文化传播机制，确定后代个体的模仿任务。
13. Evaluate the candidate individuals in CP_j^t 评估生成的后代个体的适应度。
14. End for 结束针对辅助任务的内部循环。
15. Carry out elitist selection to form $C^{t+1}, P^{t+1} = \{P_j^{t+1}\}_{j=1}^2$ 执行精英选择策略，筛选出下一代的目标种群和辅助种群。
16. Update auxiliary tasks via Algorithm 3 通过算法 3 动态更新或生成新的辅助任务。
17. Set $t = t + 1$ 迭代代数自增 1。
18. End while 结束主循环。

2.4 算法 2 伪代码

输入 (Input):

Target population - C^t 输入: 目标种群 C^t

Auxiliary populations - P^t 输入: 辅助种群 P^t

Scaling factor - γ 输入: 缩放因子 γ

输出 (Output):

The improved candidate individual of target population 输出: 优化后的目标种群候选个体

算法步骤:

1. Randomly select candidate individuals p_1, p_2 from $P^t = \{P_j^t\}_{j=1}^2$, respectively 1. 分别从两个辅助种群中随机选择候选个体 p_1 和 p_2 。
2. Randomly select a candidate individual c from C^t 2. 从目标种群 C^t 中随机选择一个候选个体 c 。
3. For $j = 1$ to 2 do 3. 对于第 1 到第 2 个辅助任务执行循环。
4. Transfer the p_j^s to the candidate individual c to obtain $c(p_j^s)$ 4. 将辅助个体的源决策变量 p_j^s 迁移至目标候选个体 c ，得到中间个体 $c(p_j^s)$ 。

5. If $c(p_j^s)$ better than c then 5. 如果中间个体 $c(p_j^s)$ 的适应度优于原个体 c 。
6. $c(p_j^s) \rightarrow c$ 6. 将原个体 c 更新为中间个体 $c(p_j^s)$ 。
7. End if 7. 结束本次条件判断。
8. End for 8. 结束源决策变量的迁移循环。
9. Extract extended decision variables p_1^e, p_2^e to generate mutation vector V^e based on Eq. (13) 9. 提取扩展决策变量 p_1^e 和 p_2^e ，并根据公式 13 生成变异向量 V^e 。
10. For each V_i^e in V^e do 10. 遍历变异向量 V^e 中的每一个分量 V_i^e 。
11. Transfer the V_i^e to the candidate individual c to obtain $c(V_i^e)$ 11. 将变异分量 V_i^e 迁移至目标候选个体 c ，得到中间个体 $c(V_i^e)$ 。
12. If $c(V_i^e)$ better than c then 12. 如果该中间个体 $c(V_i^e)$ 的适应度优于原个体 c 。
13. $c(V_i^e) \rightarrow c$ 13. 将原个体 c 更新为中间个体 $c(V_i^e)$ 。
14. End if 14. 结束本次条件判断。
15. End for 15. 结束变异向量的迁移循环。

2.5 算法 3 伪代码

输入 (Input):

Auxiliary populations at generation $t - P^t$ 第 t 代的辅助种群 P^t

Auxiliary populations at generation $t + 1 - P^{t+1}$ 第 $t + 1$ 代的辅助种群 P^{t+1}

Convergence coefficient $-\beta$ 收敛系数 β

输出 (Output):

The updated auxiliary tasks 更新后的辅助任务

算法步骤:

1. For $j = 1$ to 2 do 对于第 1 到第 2 个辅助任务执行循环。
2. Compute $\sigma_j = \text{RSME}(P_j^t, P_j^{t+1})$ based on Eq. (14) 根据公式 14 计算该辅助种群在连续两代之间的均方根误差 σ_j 。
3. If $(\sigma_j \leq \beta)$ then 如果误差 σ_j 小于或等于收敛系数 β (说明该辅助任务演化已陷入停滞或收敛)。
4. Generate new auxiliary task TA_j based on Eq. (10), (11) 根据公式 10 或 11 重新生成新的辅助任务 TA_j 。
5. Initialize auxiliary population P_j^t for TA_j 为新生成的辅助任务 TA_j 重新初始化辅助种群 P_j^t 。
6. Evaluate candidate individuals in P_j^t 评估新初始化的辅助种群中个体的适应度。
7. End if 结束条件判断。
8. End for 结束循环。